LangGraph & AI Engineering Roadmap

Project: a multi-node RAG Patient Information Assistant for a fictional clinic

How to read the estimates

Estimates assume **part-time study of about 6-8 hours per week** alongside a full-time job, and include reading docs, building, debugging and code review. At 12-15 h/week, roughly halve them. Phase 5 (RAG) is deliberately the longest: data cleaning and retrieval quality are where most real projects lose time. A phase is done when its deliverable works, not when the weeks run out.

Phase	Topic	Weeks	Deliverable
0	**Prerequisites (optional)**	1-2	A clean project skeleton: repo, venv, .env, a 'hello LLM' call that works.
1	**LangChain basics & structured output**	2	A triage/classification script that returns in_scope, request_type, topic and reasoning, tested on tricky questions.
2	**Your first graph**	2	Graph: safety filter -> LLM answer, with emergencies short-circuiting to a fixed message.
3	**Branching: the Triage node**	2	Graph: safety filter -> triage -> (answer | polite refusal | redirect to professional).
4	**Router & multiple agents**	3	Full routing pipeline working end to end with hardcoded knowledge.
5	**RAG: real documents**	5	Each agent answers from its own retrieved documents and returns its sources.
6	**Personality, Grounding & cycles**	4	The full 6-node pipeline from the original project, with a retry loop.
7	**Observability, evaluation & memory**	4	An eval suite you can re-run after every change, plus multi-turn conversations.
8	**Stretch: deployment & advanced patterns (optional)**	4-6	A deployed API with a simple frontend or Swagger UI.

Scope	Weeks	Approx. months
Core path (Phases 1-7)	~22	~5 months
Core + prerequisites + buffer for life/work	~24-26	**~6 months (realistic target)**
Including Phase 8 stretch	~28-32	~6.5-7.5 months

Starting in November 2026, the core path would finish around April-May 2027.

Timeline (weeks)

Bar widths are proportional to estimated weeks. Grey = optional stretch phase.

Phase details

Phase 0 - Prerequisites (optional) (1-2 weeks)

Make sure the Python foundations won't slow you down later.

• Virtual environments (venv / uv), dependency management, .env files for API keys
• Type hints and TypedDict (LangGraph state is built on them)
• Pydantic models: fields, validation, Literal / Enum types
• async/await basics (needed later for streaming and APIs)

Deliverable: A clean project skeleton: repo, venv, .env, a 'hello LLM' call that works.

Phase 1 - LangChain basics & structured output (2 weeks)

Talk to an LLM reliably and get typed data back instead of free text.

• Chat models (AzureChatOpenAI / ChatOpenAI) and .invoke()
• ChatPromptTemplate: system vs. user messages, variables
• Structured output with Pydantic and .with_structured_output()
• Prompt design for classification: clear categories, examples, edge cases
• Defining policies for ambiguous cases (informational vs. personal advice)

Deliverable: A triage/classification script that returns in_scope, request_type, topic and reasoning, tested on tricky questions.

Phase 2 - Your first graph (2 weeks)

Understand LangGraph's core model: state flows through nodes.

• StateGraph, state schema (TypedDict), reducers
• Nodes as plain Python functions (no LLM required)
• Edges, START / END, compile() and invoke()
• Deterministic node: offensive words + emergency red-flag detection
• Visualizing the graph (Mermaid / PNG)

Deliverable: Graph: safety filter -> LLM answer, with emergencies short-circuiting to a fixed message.

Phase 3 - Branching: the Triage node (2 weeks)

Make the graph take different paths depending on what the LLM decides.

• Conditional edges and routing functions
• Wrapping your Phase 1 classifier as a graph node
• Out-of-scope and 'see a doctor' responses
• Error handling: what if the LLM call fails or returns garbage?

Deliverable: Graph: safety filter -> triage -> (answer | polite refusal | redirect to professional).

Phase 4 - Router & multiple agents (3 weeks)

Scale to several specialized agents and design state that holds up.

• Router node and multi-way conditional edges
• Three agents: Conditions, Medications, Clinic admin (prompt-only knowledge for now)
• State design: what each node reads/writes; avoiding a 'god state'
• Design trade-off: separate Triage + Router vs. one combined call
• Basic unit tests per node

Deliverable: Full routing pipeline working end to end with hardcoded knowledge.

Phase 5 - RAG: real documents (5 weeks)

Ground each agent in real sources. Usually the longest and most underestimated phase.

• Ingestion: parsing MedlinePlus XML, openFDA JSON, your own clinic markdown
• Chunking strategies and metadata (source, topic, language)
• Embeddings and a vector store (Chroma/FAISS locally; Azure AI Search later)
• Retrievers: top-k, similarity thresholds, metadata filters per agent
• Retrieval evaluation: does the right chunk come back? (hit rate, MRR)
• Cross-language retrieval (Spanish question, English documents)

Deliverable: Each agent answers from its own retrieved documents and returns its sources.

Phase 6 - Personality, Grounding & cycles (4 weeks)

Complete the original architecture, including LangGraph's key advantage: loops.

• Personality node: voice and tone without changing facts
• Grounding node: claim-by-claim verification against retrieved context
• Cycles: if grounding fails, regenerate (with a max-retry counter)
• Safe fallback when retries are exhausted
• Placement trade-off: grounding before vs. after personality

Deliverable: The full 6-node pipeline from the original project, with a retry loop.

Phase 7 - Observability, evaluation & memory (4 weeks)

Make the system measurable and production-minded. Your QA background pays off here.

• Tracing with LangSmith (or Langfuse): inspecting every node run
• Building an evaluation dataset (golden Q&A;, adversarial cases)
• LLM-as-judge evals: faithfulness, relevance, tone, safety
• Conversation memory with checkpointers; thread IDs
• Streaming responses; cost and latency tracking per node

Deliverable: An eval suite you can re-run after every change, plus multi-turn conversations.

Phase 8 - Stretch: deployment & advanced patterns (optional) (4-6 weeks)

Take it from a script to a service, and learn patterns beyond fixed pipelines.

• Serving the graph with FastAPI; Docker
• Deploying on Azure (Container Apps / App Service, Azure OpenAI, AI Search)
• Tool-calling agents and the ReAct pattern (e.g. live openFDA lookups)
• Human-in-the-loop with interrupts
• Guardrail hardening: prompt-injection tests, PII handling

Deliverable: A deployed API with a simple frontend or Swagger UI.

Data sources

Conditions agent: MedlinePlus health topic XML (NLM) - English & Spanish, CC0. medlineplus.gov/xml.html. Start with 15-20 topics.
Medications agent: openFDA drug labels - api.fda.gov/drug/label.json (docs: open.fda.gov). Save ~10 labels locally as JSON.
Clinic admin agent: Markdown files you write yourself for a fictional clinic: known ground truth for testing grounding.

Working agreement

• Each phase: build it yourself, then share code for a senior-style review before moving on.

• Keep a short decision log (policies, trade-offs, why you chose X over Y). It becomes your portfolio write-up.
• Don't skip ahead: phases 1-4 need no documents; download data only at Phase 5.